

intensive applications. "Since the number of cores and computing power in a GPU is fixed, there is a risk of under-utilisation of or compromise on the performance or the application," explains Vasudevan. He adds, "A hardware accelerator implemented in an FPGA can be paired with a single-core or multicore device to give a very suitable silicon for certain applications." In many ways, these are all complementary technologies.

For a multicore developer

As in the case of any other product development, multicore system developers also need to consider several aspects and gain expertise in certain methodologies to effectively use the new silicon capability. When an application cannot be easily divided among multiple cores, it may not be able to fully utilise the capability of all the cores.

The greater challenge lies on the software development front to effectively harness the performance available in multicore configurations. Thronson shares, "Due to increasing performance demands, the move from single-core solutions to multicore solutions is inevitable and is already being demonstrated in a variety of embedded applications. However, optimised software for multicore lags the full capabilities of what can be achieved today in hardware. So software development and tools to simplify threaded software design are equally important."

Agreeing to this, Kumar says, "It is very critical to have proper software partitioning on the go. For example, when the load increases, the application should automatically switch between cores to deliver optimal performance." He adds, "Synchroni-



Benefits of using multicore (Image courtesy: Hexus.net)

sation is another important aspect to it. In symmetric multiprocessing, the software overhead is much less than asymmetric multiprocessing."

Pathak says, "Hardware challenges before semiconductor companies include glue between the cores and sharing the load with respect to the input signals and data between cores. This also requires a hardware design to ensure that the software can take advantage of the increased computing power. Task partitioning and handling interrupts are one of the key challenges before software engineers, especially for heterogeneous cores (where instruction set types differ from one core to another). Sharing various MCU resources (like memory, input/output and peripheral channels) is another design challenge where every signal and functionality has to meet its respective timing requirements for real-time systems."

The design must take care of inter-process communication and synchronisation. "So design engineers need to have a complete understanding of the product (system and hardware) along with skills in various OS concepts of memory management unit, task scheduling, interrupt handling, inter-process communication, synchronisation (Semaphore), etc," says Pathak.

Hardware parallelism. Programmers making the switch to multicore

need to better understand true hardware parallelism and basic parallel programming techniques. Concepts of threads, locks and parallelisation, in general, assume critical importance.

Kumaran adds, "True hardware concurrency can introduce new behavioural problems that do not exist in single-core systems. Moreover, system analysis, visualisation and understanding are more difficult in a truly parallel system."

Graphical programming.

Traditional programming languages which use a top-down sequential execution approach have inherent limitations when mapping to parallel hardware. Panikkar says, "Graphical dataflow programming models are in contrast to the control-flow model implemented in languages such as C. With a dataflow model, nodes on a block diagram are connected to one another to express the logical execution flow, and they can be used to easily express parallelism. When a block diagram node receives all the required inputs, it produces output data and passes that data to the next node in the dataflow path. The movement of data through the nodes determines the execution order of the functions on the block diagram. Parallel programming methodologies like task and data parallelism, pipelining and structure grids need to be utilised."

Embedded virtualisation. Embedded virtualisation allows running of multiple operating systems on the same processing hardware, and helps in creating the necessary software abstraction to simplify development for programmers. Justifying this with an example, Panikkar says, "Consider an embedded application with control algorithms that require a real-time OS, with a user interface and networking components that are natural for a general-purpose OS. These needs are easily met by virtualisation, which